

Concurrency Keywords and Features in Major Languages

Overview

This document compares the concurrency-related keywords and features in several programming languages and Ada versions.

Concurrency Keyword Table (Summary)

Due to layout constraints, this table is abbreviated. Full comparison included in a future section.

Languages: C++, Python, Java, Rust, Ada83, Ada95, Ada2005, Ada2012, Ada2022

- Thread creation: `std::thread`, `threading.Thread`, `Thread/Runnable`, `task`, etc.
- Protection: `mutex`, `Lock`, `synchronized`, `protected`
- Communication: `condition_variable`, `Queue`, `entry/accept`, `channels`
- Parallelism: `parallel loop`, `rayon`, `parallel case` (Ada2022)
- Async: `async/await` in Python, Rust, Ada2022

Language-Specific Highlights

C++:

- Threads via `std::thread`, manual sync
- C++20 coroutines

Python:

- `threading`, `asyncio`, GIL limits real concurrency

Java:

- Thread model, `synchronized`, `CompletableFuture`, `Loom` (upcoming)

Rust:

- Ownership-based safety, `channels`, `async/await`

Ada83:

- `task`, `entry`, `accept` for concurrency

Ada95:

- Adds protected types for safe shared access

Ada2005:

- Better real-time support, Ravenscar profile

Ada2012:

- Contracts, requeue, expressive synchronization

Ada2022:

- parallel, async/await, task groups, multicore support

Summary Table (Simplified)

Language	Concurrency	Safety	High-Level Abstractions	Parallelism	Built-in Keywords
C++	Low	Medium	Manual	Moderate	
Python	Low	High	Weak (GIL)	High	
Java	Medium	High	Strong	High	
Rust	Very High	Medium	Very Strong	Medium	
Ada*	Very High	High	Strong (2022)	Very High	